

Contribution Technique - II

(Using Cumulative DS).

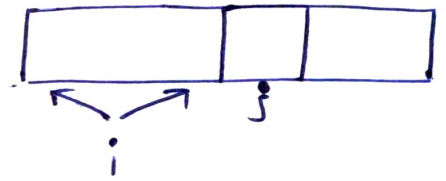
① Two Sum problem

You are given an Array (Arr) of size N and an integer X . Find the number of pairs (i, j) where $(i < j)$ & $(Arr[i] + Arr[j] = X)$.

Solⁿ : $\rightarrow$

$$Arr[i] + Arr[j] = X \quad (i < j)$$
$$Arr[i] = X - Arr[j]$$

$\rightarrow$ If we pick j as the pivot element, and if the value at j is $Arr[j]$, in the left of j , we need to find the frequency of $(X - Arr[j])$. The number of $(X - Arr[j])$ is the number of pairs for this j . Similarly, we shall do for all j .



$\rightarrow$ For frequency we will use running map.

code

```
map<int, int> mp;
for (int j=0; j<n; j++) {
    int to-find = X - arr[j];
    auto it = mp.find(X - arr[j]);
    if (it != mp.end()) {
        count += mp[X - arr[j]];
    }
    mp[arr[j]]++;
}
```

Leetcode-121 (Best Time to Buy and Sell stock).

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day. You want to maximize your profit by choosing a single day to buy one stock and choosing a diff. day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Solⁿ $\rightarrow$ We can think this problem in terms of i and j similar to the previous problem. In (j) I will sell and on (i) I will buy. So, we need to find a pair (i, j) such that $(prices[j] - prices[i])$ is maximized where `prices[j]` is the selling price and `prices[i]` represents the buying price. So, for a particular j , for $(prices[j] - prices[i])$ to be maximized, `prices[i]` must be the minimum possible, then only the difference will be maximised. $\rightarrow$ So, running through the array, we need to maintain minimum till that point (j) .

Code

```
best-profit = 0;
min-so-far = INT-MAX;
for (j = 0; j < n; j++) {
    int profit = arr[j] - min-so-far;
    best-profit = max(best-profit, profit);
    min-so-far = min(min-so-far, arr[j]);
}
```

after this
 j , this
`arr[j]` also
becomes
a potential
buying price

`cout << best-profit << "\n";`

③ You are given an array Arr of size N and an integer K . Find the smallest value of $(Arr[i] + Arr[j])$ such that $Arr[i] + Arr[j] > K$ ($i \leq j$)

Solⁿ Similar to the two-sum problem :-

$$Arr[i] + Arr[j] > K$$

$$Arr[i] > K - Arr[j]$$

$$\text{If } K - Arr[j] = X$$

then, $Arr[i] > X$

that means, for a specific j (pivot), from the bundle of previous elements (since $i < j$) I need to look for the first integer $> X$. For this we will use upper bound and set which automatically sorts the data.

Upperbound(X)
 ↳ to Find First element $> X$
 ↳ to Find # of element $\leq X$
 (only on sorted data)

code :-

```

int best = 1e9;
set<int> st;
for (int j = 0; j < n; j++) {
    (as i < j)  → st.insert(arr[j]);
    auto it = st.upper-bound(K - arr[j]);
    if (it != st.end()) {
        best = min(best, arr[j] + (*it));
    }
}
cout << best << "\n";
    
```

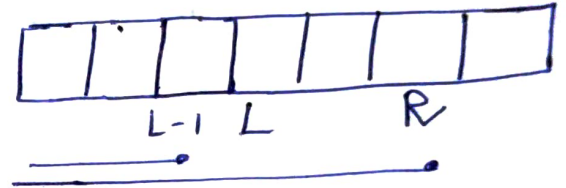
Prefix-Sums (pre Computation Technique to improve t.c)

$$A = \{4, 2, 3, 1, -5, 6\}$$

$$P = \{4, 6, 9, 10, 5, 11\}$$

Sum of subarray from index L to R =

$$P[R] - P[L-1] = \text{Sum}(L \dots R)$$



① Given an array,

0	1	0	-1	3	-2
---	---	---	----	---	----

$$X = 0$$

Find number of Subarrays whose sum = X

Solⁿ :- Any sub-array is of the form:-

$$[st, end] \rightarrow (arr[st] + arr[st+1] + \dots + arr[end]) = X$$

$$P[end] - P[st-1] == X$$

if loop on end point, -

$$P[st-1] == P[end] - X$$

no. of (L, R)
[L ≤ R] satisfying
st end

$$P[L-1] = P[R] - X$$

now, $0 \leq L \leq R$, so $-1 \leq L-1 \leq R-1$

So, in the prefix-array we will assume one extra index -1, which will have value 0.

Idea will be similar to two sum problem, where we used map, to store the frequencies.

```
int arr[n];
int p[n];
for (int i = 0; i < n; i++) {
    cin >> arr[i];
    p[i] = arr[i];
    if (i > 0) p[i] += p[i-1];
}
```

```
int ans = 0;
map<int, int> freq;
freq[0] = 1;
for (int j = 0; j < n; j++) {
    ans += freq[p[j] - x];
    freq[p[j]]++;
}
cout << ans << "\n";
```

very imp. bcz, (L-1) is also possible to be ~~zero~~ -1

Similar problem → Leetcode (560).

Let's say in the previous question, i need to print the indices as well — then code.

```
int ans = 0;
map<int, vector<int>> freq;
freq[0].push_back(-1);
for (int r = 0; r < n; r++) {
    for (auto l : freq[p[r] - x]) {
        l++;
        cout << l << " - " << r << endl;
    }
}
```

```

freq [p[r]].push-back(r);
}

```

For all potential elements, we are storing their indices.

Input

6 0

0 1 0 - 1 3 - 2

Output

0 - 0

2 - 2

0 - 3

1 - 3

2 - 5

3 - 5

0